Annexe A20 — Leçon : le dessin en point

LibraryV3 · Phase G, étape 4a · un objet plus petit qu'un pixel ne se rastérise pas, il se pose · 11 octobre 2026

En une ligne. Quand la sphère englobante d'un objet a un rayon apparent inférieur à un seuil (1 px par défaut), on ne transforme **aucun** sommet et on ne teste **aucune** face : on projette le centre de l'objet et on écrit **un seul fragment**, par le même chemin que les triangles (test de profondeur, mode de la vue). Sur la scène active : `Render` ÷ 2,9, aucun pixel perdu, et une ceinture d'astéroïdes **plus fidèle** qu'avant.

1. Le problème : ce qu'un triangle sous-pixel allume vraiment

Un rasterizer n'allume pas les pixels qu'un triangle « touche » : il allume les pixels dont le **centre**, en (i + ½, j + ½), est dans le triangle. C'est la règle top-left, verrouillée par nos TNR, qui garantit qu'un pixel n'est jamais dessiné deux fois ni oublié entre deux triangles voisins.

Pour un grand triangle, la différence est invisible. Pour un triangle de 0,4 px, elle décide de tout : s'il tombe entre quatre centres, il n'allume **rien**. S'il en recouvre un, il allume un pixel. D'une frame à l'autre, au gré de quelques centièmes de pixel de mouvement, l'objet **clignote**.

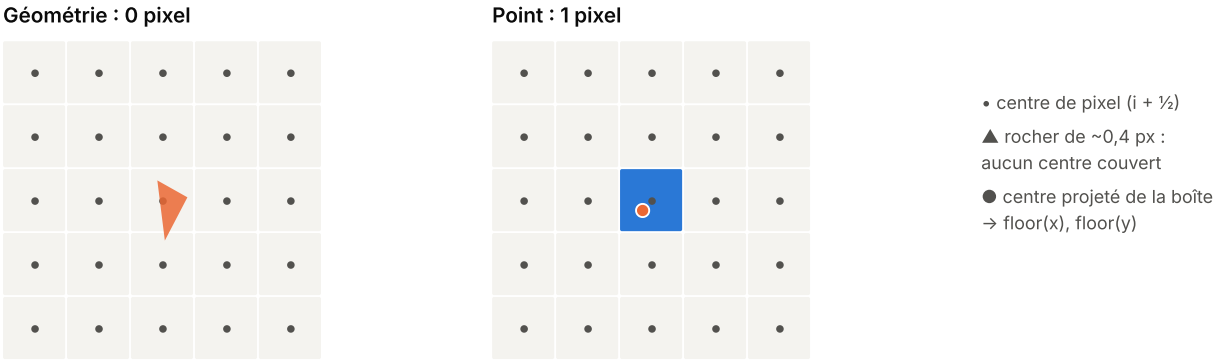


Figure 1 — Un triangle sous-pixel n'allume un pixel que s'il couvre son CENTRE (règle top-left). Entre deux centres, il disparaît : la ceinture de 600 rochers n'allumait que 99 pixels vue de dessus. Le point allume toujours le pixel qui contient le centre de l'objet.

Mesuré sur la scène active, vue de dessus : les 600 rochers de la ceinture n'allumaient que **99 pixels**. Et pour obtenir ce résultat, chaque rocher payait (A19) environ 92 ns de préparation et 10,7 faces × 14,4 ns, soit ≈ 245 ns en banc et ≈ 305 ns en situation, pour un résultat le plus souvent nul.

> **À retenir.** Un rasterizer échantillonne : il répond à la question « ce centre de pixel est-il dans le triangle ? ». En dessous de la taille d'un pixel, la géométrie n'est plus une **forme**, c'est une **probabilité de toucher un centre**. Dépenser des centaines de cycles pour tirer cette probabilité est le pire des deux mondes : cher, et instable.

2. Trois réponses possibles

Réponse	Coût	Image	Verdict
Rien changer	≈ 305 ns par objet	instable, la ceinture disparaît à moitié	×
Ne pas dessiner (rejet par contribution)	≈ 20 ns	la ceinture **disparaît**	×
Dessiner un point	≈ 20 à 30 ns	un pixel stable par objet	✓

> ✕ Le rejet par contribution de manuel (`if (rPx < 1) continue;`). Unreal (*Min Screen Radius*) et Unity le font pour des objets dont l'absence passe inaperçue : un caillou dans un décor. Ici, l'objet sous-pixel **est** la scène : 1 278 des 1 297 objets dessinés. On ne supprime pas un objet sous-pixel ; on change la façon de le dessiner.

La décision d'Onky : *si c'est pour afficher 1 ou 2 pixels de l'objet, autant afficher un point avec la même luminosité ; de loin, on ne verra aucune différence.*

3. Le principe, étape par étape

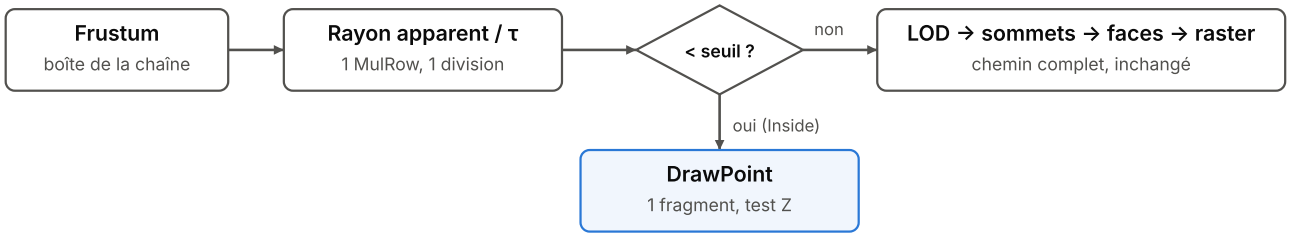


Figure 2 — Le test se place AVANT la sélection du niveau et avant toute transformation de sommet. Le rayon apparent est déjà calculé pour le LOD : le test ne coûte qu'une comparaison.

Étape 1 — Où placer le test. Après le frustum (un objet hors champ ne coûte déjà que 19 ns) et **avant** la sélection du niveau de LOD : à cet endroit, on n'a encore lu ni le mesh, ni ses sommets, ni ses indices.

Étape 2 — Quelle grandeur comparer. Le rayon apparent de la sphère englobante est **déjà calculé** pour le LOD :

```
rOverTau = ProjectedRadiusPx(view.lodParams, wC, rW)    // rayon en pixels, déjà divisé par tau
```

`lodParams.kPx` contient déjà la division par τ, faite une fois par vue dans `BuildViewData`. Plutôt que de remultiplier par τ à chaque objet, on convertit **le seuil** dans la même unité, une fois par vue :

```
view.pointRadiusOverTau = EngineConfig.lod.pointRadiusPx / view.lodTolerancePx
```

La comparaison par objet est alors une seule instruction : `rOverTau < view.pointRadiusOverTau`.

Étape 3 — Quel point. Le centre de la boîte de la chaîne, déjà transformé : `MulRow(mvp, box.Center())` servait pour son seul `w` ; on garde aussi x, y et z. `ProjectClip` le divise par w et le passe en coordonnées raster : c'est **la même fonction** que pour les sommets des triangles, donc la même convention au bit près.

Étape 4 — Quel pixel. Celui qui **contient** le point : `floor(x), floor(y)`, puisque les centres sont en i + ½. On rejette un point hors de la vue courante.

Étape 5 — Comment l'écrire. Par le fragment shader de la vue (`m_fragment`), comme un **triangle dégénéré** dont les trois sommets sont le point, avec les poids barycentriques (1, 0, 0). Le test de profondeur (reverse-Z, `GREATER`) et le mode de la vue (Solid, Depth, LinearDepth) s'appliquent sans une ligne de code de plus.

Étape 6 — Quand ne pas le faire. Seulement pour un objet classé `Inside` par le frustum : son centre a alors un `w > 0` garanti. Un objet `Intersect` (au bord de l'écran) garde la géométrie complète, cas rare. Un rayon `+inf` (l'objet touche le near) ou `NaN` rend la comparaison fausse : géométrie complète.

> ✗ Écrire le pixel directement dans le framebuffer, sans test de profondeur. Les rochers de la ceinture se verraient **à travers** Jupiter. Un point est un fragment comme un autre.

> ✗ Placer le test après le choix du niveau, ou après `ProjectPositions`. On aurait déjà payé ce qu'on voulait éviter : la lecture du mesh, la transformation de ses sommets.

> ✗ Recalculer un rayon en pixels « propre » (`ProjectedRadiusPx(ssp, …)`) pour le seul test. Une division de plus par objet, et deux formules qui pourraient diverger. On convertit le seuil, pas la mesure.

4. Le code, fichier par fichier

Configuration (`EngineSettings.h`, `EngineConfig.h/.cpp`, `engine.json`). Une clé `lod.pointRadiusPx`, défaut 1.0, garde `>= 0` et finie (0 est **légal** : il désactive le point, le comportement redevient exactement celui de la Référence K).

```
"lod": { "tolerancePx": 0.5, "pointRadiusPx": 1.0 },
```

Vue (`ViewData.h`, `System.cpp`). `float pointRadiusOverTau`, calculé dans `BuildViewData` après `lodParams`.

Rendu (`Renderer.h/.cpp`). Une nouvelle soumission, à côté de `DrawTriangle` :

```
void Renderer::DrawPoint(const RasterVertex& p, Color color)
{
    if (!IsReady()) return;
    const int32_t x = int32_t(std::floor(p.x));            // pixel qui CONTIENT le point
    const int32_t y = int32_t(std::floor(p.y));
    if (x < m_vp.x || x >= m_vp.x + m_vp.width ||
        y < m_vp.y || y >= m_vp.y + m_vp.height) return;
    if (m_mode == ERenderMode::Wireframe) { m_ctx.fb->SetPixel(x, y, color); return; }
    if (!m_fragment) return;
    m_ctx.color = color;                                    // triangle degenere : poids (1, 0,
0)
    m_ctx.z0 = m_ctx.z1 = m_ctx.z2 = p.z;
    m_ctx.invW0 = m_ctx.invW1 = m_ctx.invW2 = p.invW;
    m_fragment(x, y, BarycentricWeights{ 1.0f, 0.0f, 0.0f }, &m_ctx);
}
```

Le cœur (`RenderSystem.cpp`, `RenderView`, étape 3) :

```
const Vec4f cClip    = MulRow(mvp, box.Center());             // centre en espace clip
const float wC       = cClip.w;
const float rOverTau = ProjectedRadiusPx(view.lodParams, wC, rW);   // UNE fois : point ET LOD

if (vis == EIntersect::Inside && rOverTau < view.pointRadiusOverTau)
{
    LV3_PROF_COUNT(EProfCounter::PointSplats, 1);
    renderer.DrawPoint(ProjectClip(cClip, view.viewport), dbg ? dbg->m_color : FaceColor(0));
    continue;                                                 // aucun sommet, aucune face
}
const float qOverTau = rOverTau / box.Extent().length();      // la selection du LOD, inchangee
```

Instrumentation (`Profiler.h`). Un compteur `PointSplats`, ajouté **en fin** d'énumération pour que les colonnes des CSV de référence ne bougent pas, et un invariant qui le relie aux autres :

```
Σ LodLevel* + PointSplats == MeshesTested − MeshesCulled
```

5. Vérifier avant de mesurer : l'image

L'image ne doit pas changer de façon visible ; mais « je ne vois pas de différence » ne prouve rien. On a donc compilé le moteur hors Visual Studio, chargé la scène active et rendu chaque caméra **deux fois** (seuil 0, puis seuil 1), puis comparé les deux images pixel par pixel :

Vue (1136 × 800)	Pixels allumés, géométrie	Pixels allumés, points	Différents
Top_View	99	159	60, tous **gagnés**
Side_View	68	68	1 (couleur)
FPS_Camera	30 216	30 216	**0**

Aucun pixel perdu. Les pixels gagnés sont ceux des rochers que la géométrie ratait. Les TNR top-left restent vertes : le chemin des triangles n'a pas bougé.

Et puisque l'image ne permet pas de savoir si le point est **actif**, c'est le compteur qui le dit : `PointSplats` vaut 1 277 à 1 282 dans la Référence L, exactement le nombre d'objets que le comptage classait sous 1 px. Un compteur à 0 aurait signalé une clé non lue (l'exe lit l'`engine.json` de **son** dossier).

6. Le résultat

`Render` : 569–618 µs → **201–209 µs**. Faces soumises : 14 752 → 1 388. Sommets transformés : 20 453 → 1 352. Le détail de la mesure, les contrôles M3 et l'écart entre prévision (314 µs) et gain réel (≈ 390 µs) sont dans A19 § 6.

7. Les limites, et ce qui viendra

Limite	Ce qu'elle implique	Quand
Couleur `FaceColor(0)`	il n'y a pas encore d'éclairage : la palette de debug sert de couleur	L08 : couleur diffuse du matériau ; L10 : même éclairage que les faces (normale = direction vers la caméra)
Profondeur du **centre**	le point est un peu derrière la surface avant de l'objet, d'au plus son rayon : quelques millièmes d'unité pour un objet sous-pixel	sans effet visible ; à garder en tête pour les ombres (L12)
Rayon **surestimé**	la sphère circonscrite à la boîte surestime d'au plus √3 : un objet passe en point un peu **plus tard** que nécessaire	sens sûr, voulu
Un pixel, plein	un objet de 0,2 px allume un pixel entier, autant qu'un objet de 1,8 px	une couverture pondérée (le pixel mélangé selon l'aire) viendra avec la transparence
Seuil par vue	les vues titre (100 px de haut) passent en point plus tôt que les grandes vues	voulu : c'est l'écran qui décide

8. Les contre-exemples, rassemblés

> × Rejeter les objets sous-pixel : la ceinture disparaît (§ 2).

> × Tester trop tard, après le LOD ou les sommets (§ 3).

> × Recalculer la mesure au lieu de convertir le seuil (§ 3).

> × Écrire le pixel sans passer par le fragment : pas de test de profondeur, pas de mode Depth (§ 3).

> × Juger à l'œil qu'une optimisation « ne change rien » : on compare les images pixel à pixel, et on vérifie par un compteur qu'elle est active (§ 5).

9. Aide-mémoire

Élément	Valeur
Clé	`engine.json` → `lod.pointRadiusPx` (défaut 1.0 ; 0 = désactivé)
Condition	`Inside` **et** `rOverTau < pointRadiusOverTau`
Coût d'un point	1 `MulRow`, 1 division, 1 fragment
Compteur	`PointSplats` ; invariant `Σ LodLevel + PointSplats = testés − rejetés`
Effet mesuré	Render ≈ 595 → 204 µs (Référence L)